

Теория автоматов

Теория автоматов и формальных языков
Лекция №6

Понятие автомата

- Автомат - это алгоритм, определяющий некоторое множество Σ , возможно, преобразующий его в другое множество.

Неформальное описание автоматов выглядит следующим образом: автомат имеет входную ленту, управляющее устройство с конечной памятью для хранения номера состояния, а также может иметь вспомогательную (рабочую) и выходную ленты.

Типы автоматов

- 1) *распознаватели* - автоматы без выхода, которые распознают, принадлежит ли входная цепочка заданному множеству L ;
- 2) *преобразователи* - автоматы с выходом, которые преобразуют входную цепочку x в цепочку y при условии, что $x \in L$.

Составные элементы автомата

- *Входную ленту* можно рассматривать как линейную последовательность ячеек, причем каждая ячейка может хранить один символ из некоторого конечного входного алфавита.
- *Входная головка* в каждый момент времени читает (обозревает) одну ячейку входной ленты.
- *Рабочая лента* - это вспомогательное хранилище информации. Данные с нее могут читаться автоматом, могут и записываться на нее.
- *Управляющее устройство* - это программа, управляющая поведением автомата.

Действия автомата

Автомат работает, выполняя последовательность рабочих тактов:

1. входная головка сдвигается вправо или стоит на месте;
2. на рабочую ленту записывается некоторая информация;
3. изменяется состояние управляющего устройства;
4. на выходную ленту (если она есть) записывается СИМВОЛ.

Конфигурация автомата

Поведение автомата удобно описывать в терминах конфигурации автомата:

- а) состояние управляющего устройства;
- б) содержимое входной ленты с положением входной головки;
- в) содержимое рабочей ленты вместе с положением рабочей головки;
- г) содержимое выходной ленты, если она есть.

Детерменированные и недетерменированные УУ

- *Управляющее устройство* может быть недетерминированным.
- В таком случае для каждой конфигурации существует конечное множество возможных следующих тактов, любой из которых автомат может сделать, исходя из этой конфигурации.
- *Управляющее устройство* будет детерминированным, если для каждой конфигурации будет возможен только один следующий такт.
- Конечные автоматы называют детерменированными (ДКА) или недетерминированными (НКА).

Типы автоматов

Существуют следующие типы автоматов:

- машина Тьюринга (МТ);
- линейно-ограниченный автомат (ЛОА);
- автомат с магазинной памятью (МП-автомат);
- конечный автомат (КА).

Сложность автомата

Сложность рабочей ленты определяет сложность автомата.

- машина Тьюринга имеет неограниченную в обе стороны ленту;
- у линейно-ограниченного автомата длина рабочей ленты представляет собой линейную функцию длины входной цепочки;
- у МП-автомата рабочая лента работает по принципу магазина LIFO;
- у конечного автомата рабочая лента отсутствует.

Формальное определение автомата

Неинициальный автомат - это пятерка вида $A = (K, X, Y, \delta, \gamma)$, где:

- K - множество состояний (алфавит состояний);
- X - входной алфавит;
- Y - выходной алфавит;
- δ - функция переходов, задающая отображение $K \cdot X \rightarrow K$;
- γ - функция выходов, задающая отображение $K \cdot X \rightarrow Y$.

Команды автомата

- Функционирование автомата можно задать множеством команд вида $qx \rightarrow ru$, где q и $r \in K$, $x \in X$, $u \in Y$.
- Пусть на некотором такте t_i управляющее устройство находится в состоянии q , а из входной ленты читается символ x .
- Если в множестве команд есть команда $qx \rightarrow ru$, то на такте t_i на выходную ленту записывается символ u , а к следующему такту t_{i+1} управляющее устройство перейдет в состояние r :

$$y(t) = \gamma(q(t), x(t)), \quad q(t+1) = \delta(q(t), x(t)).$$

Инициальный автомат

- В соответствии с определением неинициального автомата в начальный момент состояние автомата может быть произвольным.
- Если зафиксировано некоторое начальное состояние, то такой автомат называют инициальным, т.е. $q(0) = q_0$.
- Инициальный автомат - это шестерка вида $A = (K, X, Y, \delta, \gamma, q_0)$, где
- K - множество состояний (алфавит состояний);
- X - входной алфавит;
- Y - выходной алфавит;
- δ - функция переходов (отображение $K \cdot X \rightarrow K$);
- γ - функция выходов (отображение $K \cdot X \rightarrow Y$);
- q_0 - начальное состояние.

Грамматический разбор

Задача грамматического разбора заключается в нахождении вывода цепочки в заданной грамматике и определения дерева вывода этой цепочки.

Языки могут быть заданы двумя способами:

1. грамматиками (порождающее средство языка);
2. автоматами (распознающее средство языка).

Конечный автомат

Конечный автомат имеет входную ленту, с которой за один такт может быть считан один входной символ. Возврат по входной ленте не допускается.

Конечным автоматом называется пятерка вида $A = (K, \Sigma, \delta, p_0, F)$, где:

- K - конечное множество состояний;
- Σ - алфавит;
- δ - функция переходов;
- p_0 - начальное состояние;
- F - множество заключительных состояний.

Регулярные множества

- Пусть Σ - некоторый алфавит. Регулярное множество в алфавите Σ определяется рекурсивно следующим образом:
 1. $\emptyset$ - пустое множество;
 2. $\{\epsilon\}$ - множество из пустой цепочки;
 3. $\{a\}$ - регулярное множество для каждого элемента $a \in \Sigma$;
 4. если P и Q - регулярные множества в алфавите Σ , то регулярными являются множества:
 - a) $P \cup Q$
 - b) PQ
 - c) P^* .

Регулярные множества

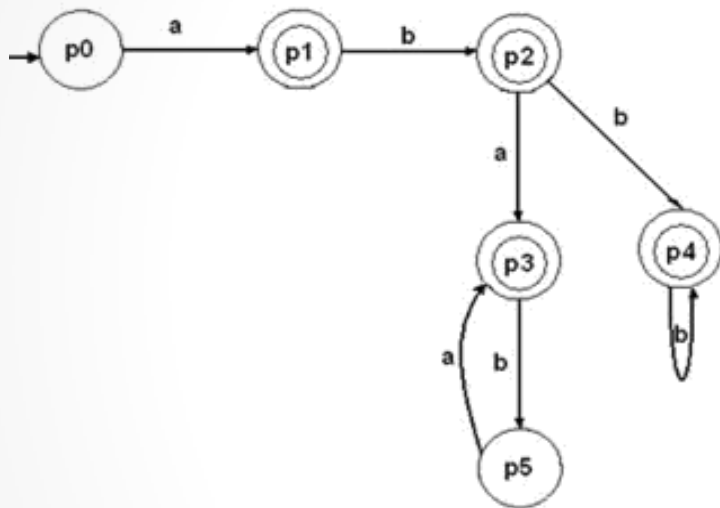
- Язык, распознаваемый конечным автоматом, - это множество цепочек, читаемых автоматом при переходе из начального состояния в одно из заключительных состояний:
- $L(A) = \{a_1 a_2 \dots a_n \mid p_0 a_1 \rightarrow p_1, p_1 a_2 \rightarrow p_2, \dots, p_{n-1} a_n \rightarrow p_n; p_n \in F\}$.
- Множество называется регулярным, если существует конечный детерминированный автомат, распознающий его.

Операции над регулярными языками

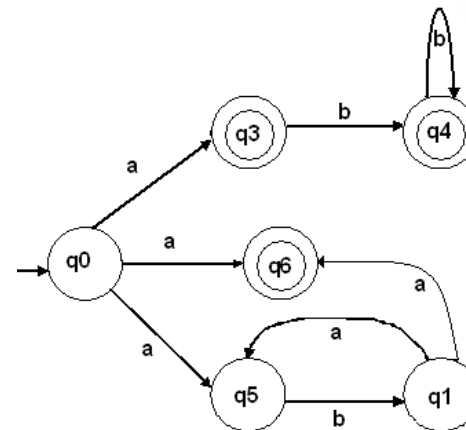
Операции над конечными автоматами эквивалентны операциям над регулярными множествами, или регулярными языками.

- Известно, что для произвольного конечного автомата можно построить эквивалентный автомат без циклов в начальных и (или) конечных состояниях.
- Множество регулярных языков замкнуто относительно операций итерации, усеченной итерации, объединения, произведения, пересечения, дополнения и разности.

Графы переходов конечных автоматов



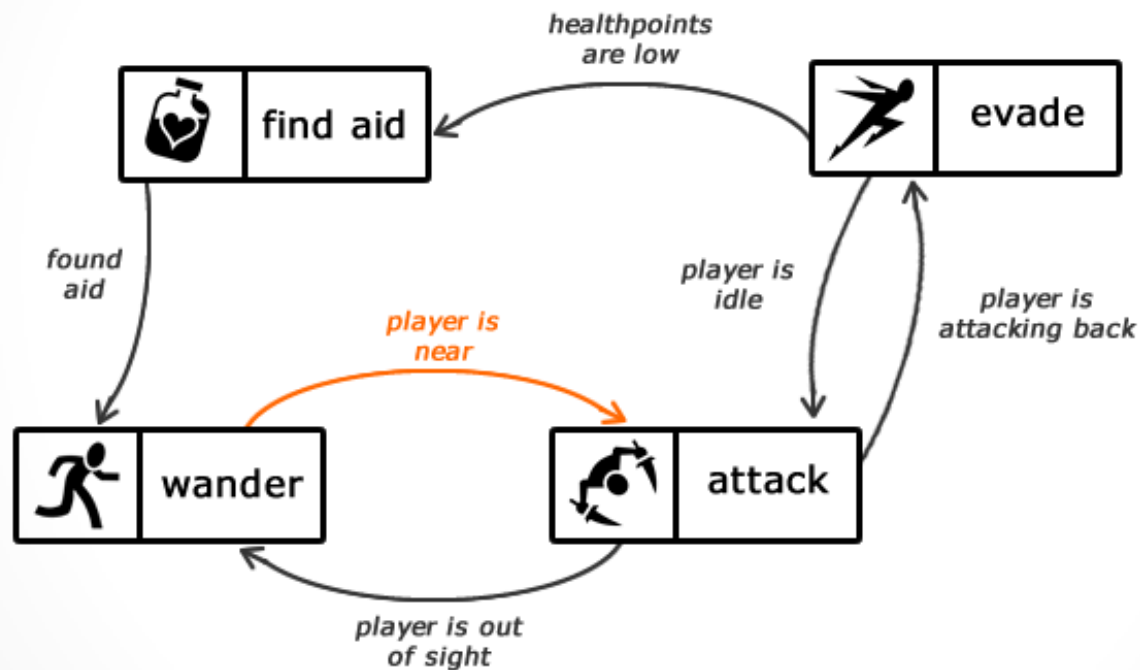
ДКА (детерминированный конечный автомат)



НКА (недетерминированный конечный автомат с переходами из одного состояния помеченными одним и тем же символом)

Конечные автоматы и примеры реализации

Автомат, описывающий поведение ИИ в игре:



- <https://tproger.ru/translations/finite-state-machines-theory-and-implementation/>

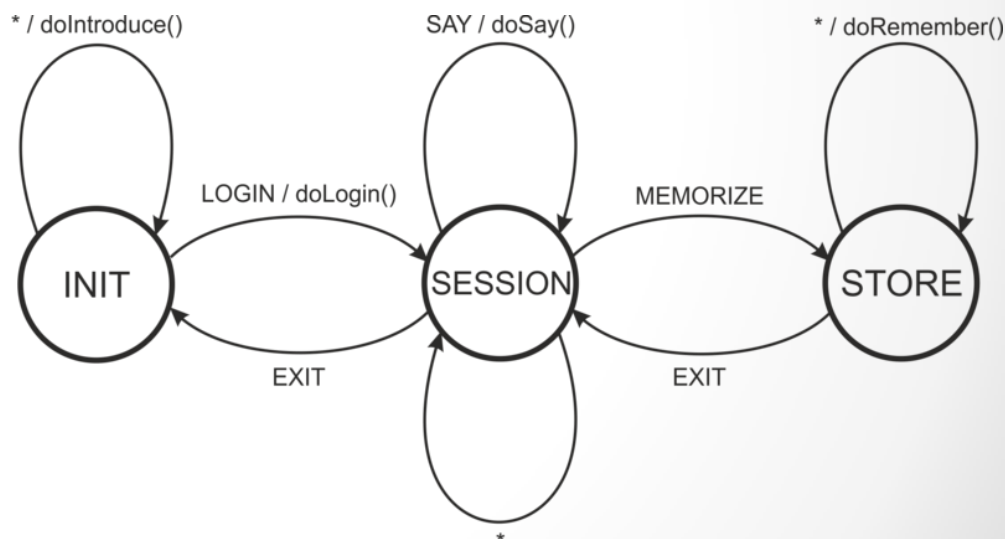
Конечные автоматы и примеры реализации

- Автомат, описывающий поведение простейшего чат-бота

Таблица переходов

Символ	Состояние	Следующее состояние	Действие
LOGIN	INIT	SESSION	Открываем сессию
*	INIT	INIT	-
*	SESSION	SESSION	-
SAY	SESSION	SESSION	Выводим строку номер N
EXIT	SESSION	INIT	-
MEMORIZE	SESSION	STORE	-
*	STORE	STORE	Сохраняем строку в буфер
EXIT	STORE	SESSION	-

Граф конечного автомата



Конечные автоматы и примеры реализации

- Конечный автомат может быть заменой громоздкого описания циклов и условий в виде `while`, `switch` вложенных конструкций.

Таблица переходов
автомата на C:

```
enum states FSM_table[3][3] = {  
    [initial][sign0] = state_1,  
    [initial][sign1] = initial,  
    [initial][sign_N] = state_1,  
    [state_1][sign0] = initial,  
    [state_1][sign1] = state_1,  
    [state_1][sign_N] = state_final,  
    [state_final][sign0] = initial,  
    [state_final][sign1] = initial,  
    [state_final][sign_N] = initial };
```

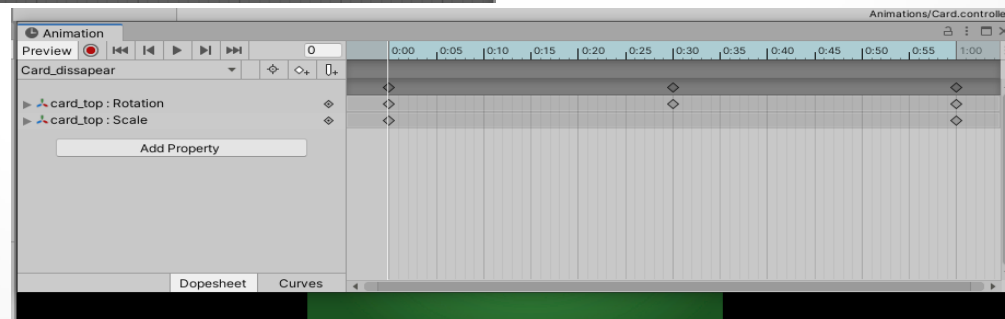
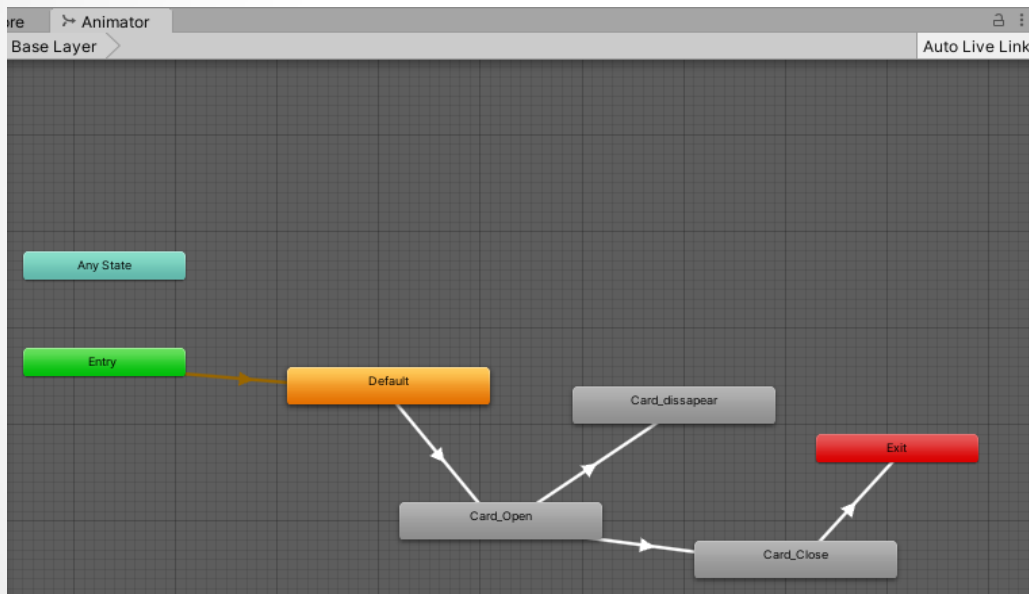
Функция обработки
переходов автомата:

```
void doFSM_table()  
{  
    enum states current_state = initial;  
    while(1)  
    {  
        enum signals current_signal = getSignal();  
        current_state =  
            FSM_table[current_state][current_signal];  
    }  
}
```

- <https://habrahabr.ru/post/241941/>

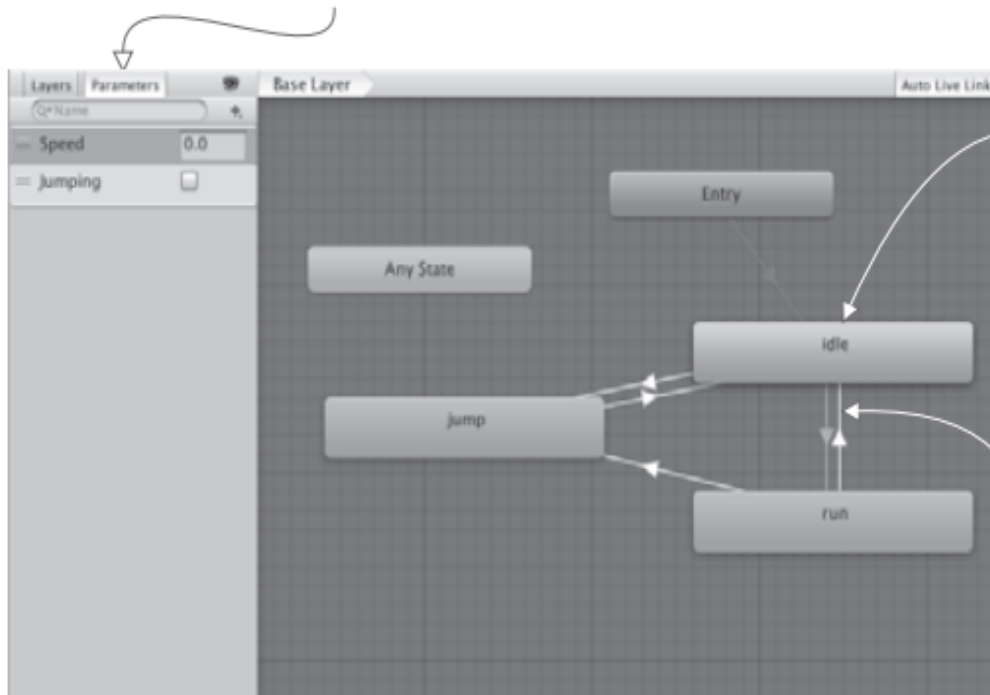
Конечные автоматы и примеры реализации

- Применение конечных автоматов для управления анимацией в Unity:



Применение конечных автоматов для управления анимацией в Unity

Здесь может быть создан набор численных или логических значений, управляющих анимацией. Переход из активного в данный момент состояния совершается при изменении этих значений



Каждый узел графа представляет собой некое состояние анимации. Как только контроллер попадает в это состояние, начинается воспроизведение указанного клипа.

(Оранжевый узел соответствует состоянию анимации до начала переходов, предлагаемому по умолчанию.)

Соединяющие узлы линии называются «переходами». Они имеют направление, например переход из состояния А в состояние В

Автоматные грамматики

Теория автоматов и формальных языков

Лекция №7

Автоматные грамматики

Линейные грамматики (праворекурсивные и леворекурсивные) называются автоматными грамматиками, так как языки, порождаемые ими, совпадают с языками, распознаваемыми конечными автоматами.

Для каждой праволинейной и леворекурсивной грамматики существует эквивалентный конечный автомат.

Для произвольного конечного автомата существует эквивалентная праволинейная и леворекурсивная грамматика.

Типы автоматов

- Автоматы с магазинной памятью:
 - Восходящий разбор в МП-автомате;
 - Нисходящий разбор в МП-автомате.
- Преобразователи.
- Автоматы Мура и Мили:
 - Восходящий разбор в МП-автомате;
 - Нисходящий разбор в МП-автомате.

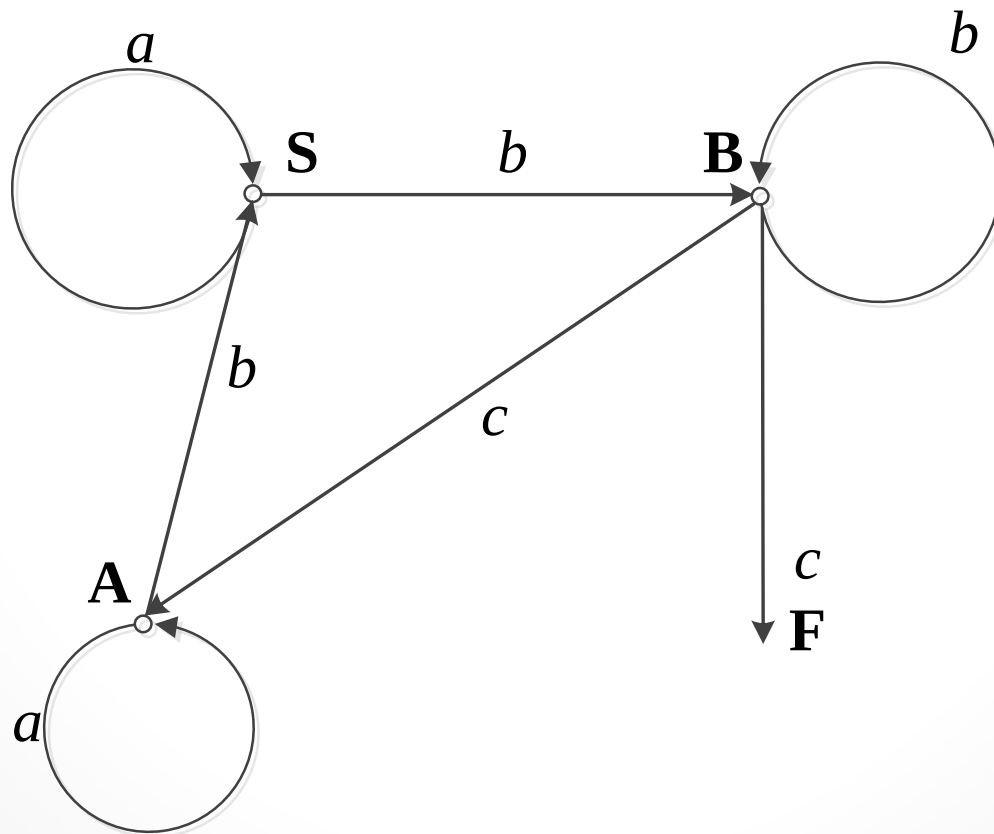
Построение конечного автомата по порождающей грамматике

- Пусть для заданной грамматики G требуется построить конечный автомат.
- $S \rightarrow aS \mid bB; A \rightarrow aA \mid bS; B \rightarrow bB \mid c \mid cA.$
- Каждому правилу грамматики поставим в соответствие команду конечного автомата:

- | | |
|----------------------|--------------------|
| • $S \rightarrow aS$ | $Sa \rightarrow S$ |
| • $S \rightarrow bB$ | $Sb \rightarrow B$ |
| • $B \rightarrow bB$ | $Bb \rightarrow B$ |
| • $B \rightarrow c$ | $Bc \rightarrow F$ |

Построение конечного автомата по порождающей грамматике

- Граф конечного автомата:



Существование эквивалентной конечному автомату грамматики

- *Теорема.* Для произвольного конечного автомата существует эквивалентная праволинейная грамматика.
- *Доказательство.* Каждому состоянию произвольного автомата поставлен в соответствие нетерминал грамматики, причем начальному состоянию будет соответствовать аксиома.
- Тогда для каждой команды $Ac \rightarrow Bb$ множество правил грамматики включим правило $A \rightarrow cB$, причем в случае, если B - заключительное состояние, добавим правило $A \rightarrow c$.

Построение конечного автомата для леворекурсивной грамматики

- Для каждой леворекурсивной грамматики существует эквивалентный конечный автомат.
- Каждому правилу грамматики $A \rightarrow Ba$ поставим в соответствие команду автомата $Ba \rightarrow A$.

Пример построения конечного автомата

- Любую автоматную грамматику можно представить в виде соответствующего конечного графа.
- По графу грамматики легко отыскивается вывод нужной цепочки.
- Любой вывод цепочки в автоматной грамматике соответствует пути в графе этой грамматики, который начинается из вершины S (вершины, помеченной аксиомой) и заканчивается в конечной вершине.

Пример построения конечного автомата

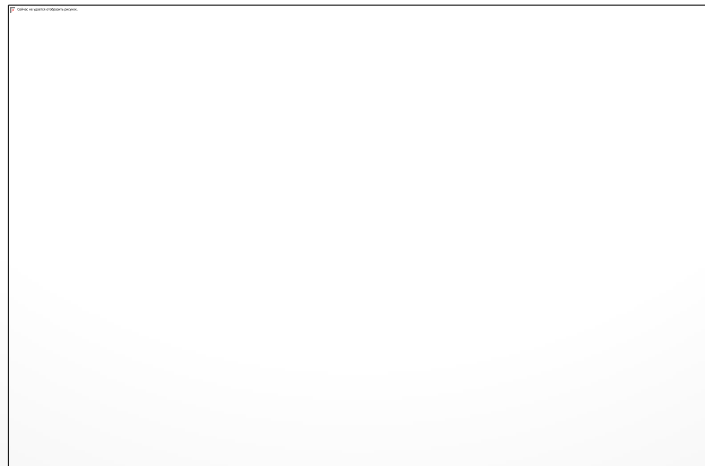
Пример. Построить конечный автомат, распознающий язык $L(A) = \{(ab)^m, m > 0\}$.

Сначала построим некоторую грамматику G , которая бы порождала язык $L(A)$:

- $S \rightarrow aA$;
- $A \rightarrow bS \mid b$.

Пример построения конечного автомата

- 1) $S \Rightarrow aA \Rightarrow ab$;
- 2) $S \Rightarrow aA \Rightarrow abS \Rightarrow abaA \Rightarrow abab$;
- 3) $S \Rightarrow aA \Rightarrow abS \Rightarrow abaA \Rightarrow ababS \Rightarrow ababaA \Rightarrow ababab$;
и т.д.
- Введем заключительное состояние F , начальное состояние соответствует аксиоме S :



Пример построения конечного автомата

Запишем преобразование правил вывода в команды:

- $Sa \rightarrow A$ - из состояния S при поступлении на вход терминала a автомат переходит в состояние A ;
- $Ab \rightarrow S$ - из состояния A при поступлении на вход терминала a автомат переходит в состояние S ;
- $Ab \rightarrow F$ - из состояния A при поступлении на вход терминала b автомат переходит в заключительное состояние F .

Таким образом, построен недетерминированный конечный автомат, распознающий заданный язык $L(G)$.

Автоматы с магазинной памятью

Автомат с магазинной памятью (МП-автомат) имеет рабочую ленту, которая организована в виде магазина.

МП-автомат - это семерка вида: $M = (K, \Sigma, \Gamma, \delta, p_0, F, B_0)$, где

- K - конечное множество состояний;
- Σ - алфавит;
- Γ - алфавит магазина;
- δ - функция переходов;
- p_0 - начальное состояние;
- F - множество заключительных состояний;
- B_0 - символ из множества Γ для обозначения маркера дна магазина.

Ограничения конечных автоматов

Что может конечный автомат?

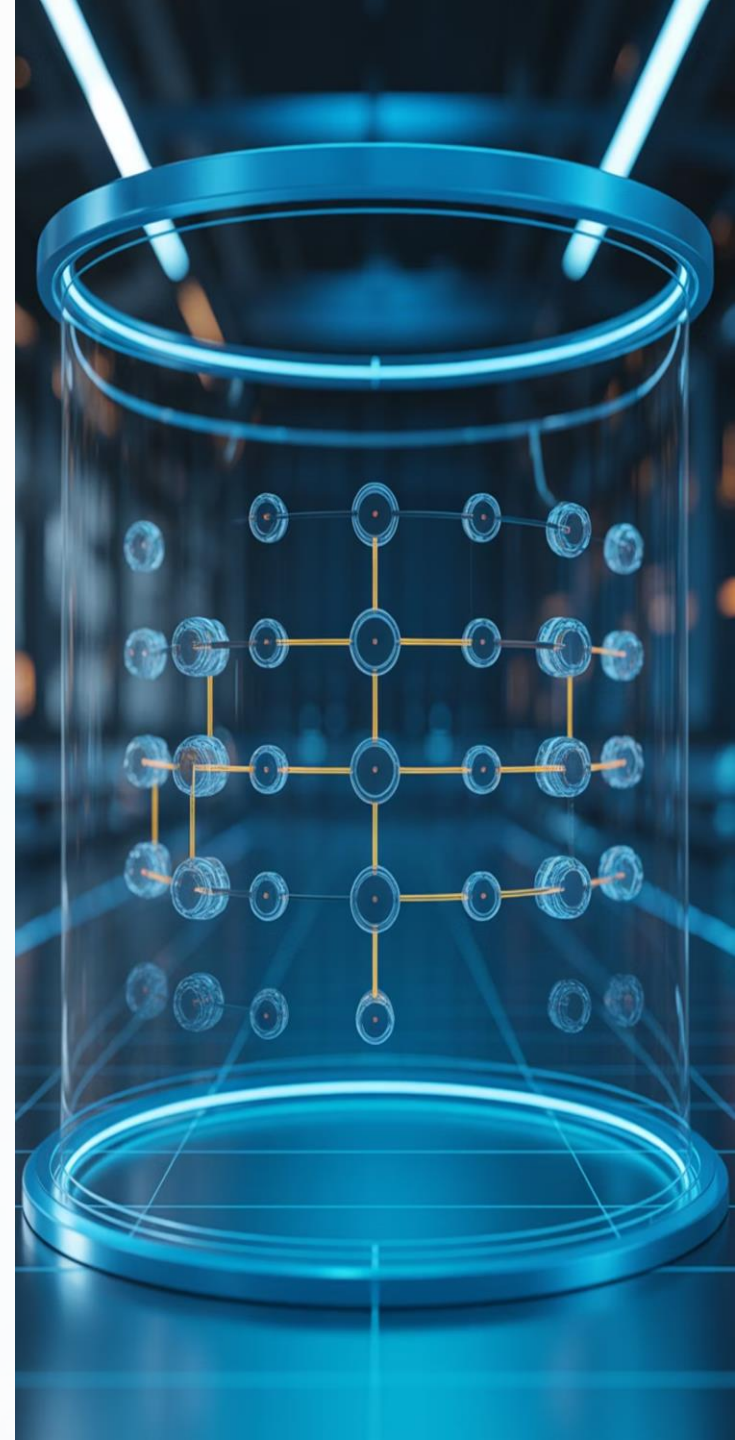
Конечный автомат (КА) — элегантная и эффективная модель вычислений для распознавания регулярных языков. Он отлично справляется с задачами, где не требуется запоминать неограниченное количество информации.

- Распознавание простых шаблонов (a^*b^*)
- Проверка делимости чисел
- Лексический анализ токенов
- Поиск подстрок в тексте

Фундаментальное ограничение

Память конечного автомата ограничена его текущим состоянием. У КА нет "истории" — он не может запомнить произвольное количество информации о прошлых событиях. Число состояний всегда конечно.

Критический пример: КА не может подсчитать неограниченное количество символов и затем проверить точное соответствие.



Классический пример: Язык $L = \{a^n b^n \mid n \geq 0\}$

Рассмотрим язык, где за последовательностью символов 'a' следует точно такая же последовательность символов 'b'. Это контекстно-свободный язык, демонстрирующий границы возможностей конечных автоматов.

Примеры цепочек

- ϵ (пустая строка)
- ab
- aabb
- aaabbb
- aaaabbbb

Не принадлежат языку

- a (нет соответствующих b)
- aab (неравное количество)
- abab (чередование)
- bbaaa (неправильный порядок)

Проблема для КА: Как запомнить количество символов 'a' (которое может быть сколь угодно большим), чтобы затем проверить соответствующее количество 'b', если число состояний конечно?

Ответ: никак. Конечному автомату не хватает вычислительной мощности. Нужна дополнительная память, но не такая универсальная, как у машины Тьюринга.

Магазинный автомат:

Определение и структура

Магазинный автомат (Pushdown Automaton, PDA) — это конечный автомат, расширенный структурой данных типа стек (магазин). Эта дополнительная память работает по принципу LIFO (Last In — First Out): последний пришел — первый ушел.

Формальное определение

МП-автомат — это семёрка $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, где каждый компонент играет свою роль в распознавании языка.

Принцип работы стека

Операции возможны только с верхним элементом:

- **Push** — поместить символ на вершину
- **Pop** — удалить символ с вершины
- **Top** — прочитать верхний символ



Построение МП-автомата для $L = \{a^n b^n \mid n \geq 0\}$

Построим конкретный магазинный автомат, допускающий цепочки по пустому стеку. Этот пример иллюстрирует, как стек используется для "запоминания" количества символов.

Компоненты автомата

- $Q = \{q_0, q_1\}$
- $\Sigma = \{a, b\}$
- $\Gamma = \{A, Z_0\}$
- Начальное состояние: q_0
- Начальный символ магазина: Z_0
- $F = \emptyset$ (допуск по пустому стеку)

Идея алгоритма

Фаза 1 (состояние q_0): Читаем все символы 'a' и помещаем в стек маркер A для каждого прочитанного символа.

Фаза 2 (состояние q_1): Читаем символы 'b' и удаляем по одному маркеру A из стека для каждого 'b'.

Проверка: Если стек становится пустым — количество 'a' и 'b' совпадает!

Функция переходов δ : Пять ключевых правил



$$\delta(q_0, a, Z_0) = \{(q_0, AZ_0)\}$$

Ситуация: В начальном состоянии видим первую 'a' на дне стека Z_0 .

Как это работает для цепочки **aabb** :

Текущее состояние	Входной символ	Верхушка стека	Действие (по функции δ)	Новое состояние	Новое содержимое стека
q0	a	Z0	Применить правило 1	q0	A Z0
q0	a	A	Применить правило 2	q0	A A Z0
q0	b	A	Применить правило 3	q1	A Z0
q1	b	A	Применить правило 4	q1	Z0
q1	(конец)	Z0	Применить правило 5	q1	ϵ (пусто)

Стек пуст $\rightarrow$ цепочка **aabb** допущена!



$$\delta(q_1, \epsilon, Z_0) = \{(q_1, \epsilon)\}$$

Ситуация: Вход прочитан полностью (ϵ -переход), в стеке остался только Z_0 .

Действие: Удаляем Z_0 . Стек пуст $\rightarrow$ цепочка допускается! ✓

Как это работает для цепочки `aabb` :

Текущее состояние	Входной символ	Верхушка стека	Действие (по функции δ)	Новое состояние	Новое содержимое стека
<code>q0</code>	<code>a</code>	<code>z0</code>	Применить правило 1	<code>q0</code>	<code>A z0</code>
<code>q0</code>	<code>a</code>	<code>A</code>	Применить правило 2	<code>q0</code>	<code>A A z0</code>
<code>q0</code>	<code>b</code>	<code>A</code>	Применить правило 3	<code>q1</code>	<code>A z0</code>
<code>q1</code>	<code>b</code>	<code>A</code>	Применить правило 4	<code>q1</code>	<code>z0</code>
<code>q1</code>	(конец)	<code>z0</code>	Применить правило 5	<code>q1</code>	<code>ε</code> (пусто)

Стек пуст → цепочка `aabb` допущена!

Грамматический разбор при помощи МП-автоматов

- Основное использование распознавательных средств задания языков состоит в построении алгоритмов *грамматического разбора*.
- МП-автомат представляет интерес как средство разбора в КС-грамматиках произвольного вида.
- Языки, порождаемые КС-грамматиками, совпадают с языками, распознаваемыми МП-автоматами.

Восходящий разбор в МП-автомате

- При восходящей стратегии необходимо найти основу и редуцировать ее к какому-нибудь нетерминалу в соответствии с правилами данной грамматики.
 - а) любой входной символ записывается в магазин;
 - б) если в верхушке магазина сформирована основа, совпадающая с правой частью правила, то она заменяется на нетерминал в левой части этого правила;
 - в) разбор заканчивается, если в магазине остается аксиома, а входная цепочка рассмотрена полностью.

Восходящий разбор в МП-автомате

Пример. Пусть дана грамматика G :

- $S \rightarrow S+A \mid S/A \mid A$
- $A \rightarrow a \mid (S);$
- $V_N = \{S, A\}, V_T = \{a, (,), +, /\}.$
- Для заданной КС-грамматики G необходимо построить МП-автомат при помощи восходящей стратегии разбора. Эквивалентный МП-автомат должен содержать следующие команды:

Команды занесения терминальных СИМВОЛОВ В МАГАЗИН

1. Команды переноса терминальных символов в магазин:

- $p_0, a, \varepsilon \rightarrow p_0, a;$
- $p_0, +, \varepsilon \rightarrow p_0, +;$
- $p_0, /, \varepsilon \rightarrow p_0, /;$
- $p_0, (, \varepsilon \rightarrow p_0, (;$
- $p_0,), \varepsilon \rightarrow p_0,).$

Команды редукции по правилам грамматики

2. Команды редукции по правилам грамматики:

- $p_0, \varepsilon, A+S \rightarrow p_0, S;$
- $p_0, \varepsilon, A/S \rightarrow p_0, S;$
- $p_0, \varepsilon, A \rightarrow p_0, S;$
- $p_0, \varepsilon, (S) \rightarrow p_0, A;$
- $p_0, \varepsilon, a \rightarrow p_0, A.$

Команды завершения разбора

3. Команды проверки на завершение разбора:

- $p_0, \varepsilon, SB_0 \rightarrow f, B_0$.
- Разбор завершается, если в магазине остались аксиома и маркер дна магазина, а входная цепочка полностью рассмотрена.

Процесс восходящего разбора в МП-автомате

- Подадим на вход автомата цепочку $a/(a+a)$ и выполним разбор.
- Процесс разбора представлен в таблице:

Вход	$a \quad / \quad (\quad a \quad + \quad A \quad)$																
магазин	B_0	a B_0	A B_0	S B_0	$/$ S B_0	$($ $/$ S B_0	a $($ $/$ S B_0	A $($ $/$ S B_0	S $($ $/$ S B_0	$+$ S $($ $/$ S B_0	a $+$ S $($ $/$ S B_0	A $+$ S $($ $/$ S B_0	S $($ $/$ S B_0	$)$ S $($ $/$ S B_0	A $/$ S B_0	S B_0	B_0
Состояние p	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	p_0	f

Нисходящий разбор в МП-автомате

- На любом шаге нисходящего разбора должно применяться какое-либо правило. В начальный момент таким нетерминалом является аксиома.
1. в начальный момент времени в магазин заносится аксиома: $p_0, \varepsilon, \varepsilon \rightarrow p_1, S$;
 2. для каждого правила $A \rightarrow \phi \in P$ нетерминал в верхушке магазина заменяется на правую часть правила с помощью команды: $p_1, \varepsilon, A \rightarrow p_1, \phi$;
 3. для каждого терминала $a \in V_T$ выполняется сравнение символа на входной ленте с символом в верхушке магазина и его поглощение: $p_1, a, a \rightarrow p_1, \varepsilon$;
 4. разбор заканчивается по команде: $p_1, \varepsilon, B_0 \rightarrow f, B_0$.

Команды МП-автомата при нисходящем разборе

1. команда занесения аксиомы в магазин.
2. команды замены нетерминала правой частью правила.
3. команды сравнения и поглощения символа с входной ленты и символа в верхушке магазина.
4. команда завершения разбора.

Нисходящий разбор в МП-автомате

- Нисходящий разбор в МП-автомате для цепочки $a/(a+a)$ и выполним разбор.
- Процесс разбора представлен в таблице:

Вход	$a \quad / \quad (\quad a \quad + \quad a \quad)$																
магазин	B_0	S B_0	S $/$ A B_0	a $/$ A B_0	a $/$ A B_0	$/$ A B_0	A B_0	$($ S $)$ B_0	S $)$ B_0	S $+$ A $)$ B_0	A $+$ A $)$ B_0	a $+$ A $)$ B_0	$+$ A $)$ B_0	A $)$ B_0	a $)$ B_0	$)$ B_0	B_0
Состояние p	p_0	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	p_1	f

Назначение МП-автоматов

- Рассмотренные выше МП-автоматы работают недетерминированно, то есть если цепочка принадлежит языку, порождаемому заданной грамматикой, то какой-то из вариантов функционирования автомата осуществит правильный разбор.
- Если цепочка не принадлежит языку, то никакой из вариантов разбора не приведет к цели.

Понятие преобразователей

- Автоматы с выходом называются *преобразователями*.

Конечным преобразователем называется шестерка вида

- $P = (K, X, Y, f, g, q_0)$, где
- K - конечное множество состояний;
- X - входной алфавит;
- Y - выходной алфавит;
- f - функция переходов;
- g - функция выходов;
- q_0 - начальное состояние.

Автоматы Мили и Мура

- Автоматы *Мили* и *Мура* являются неинициальными автоматами.
- В отображении $S(x) = g(q_0, x)$ зафиксируем начальное состояние q_0 , в котором автомат находится в начальный момент времени.
- Начальное состояние существенно влияет на процесс конечного преобразования, т.к. определяет не только результирующую цепочку, но и множество входных цепочек.

Автомат Мили

Автомат Мили - это пятерка вида $M = (K, X, Y, f, g)$, где:

- K - множество состояний автомата;
- X - входной алфавит;
- Y - выходной алфавит;
- f - функция переходов (отображение $K \cdot X \rightarrow K$);
- g - функция выходов (отображение $K \cdot X \rightarrow Y$).

Автомат Мили

- Как и любой другой автомат, автомат Мили можно представить в виде таблицы или графа.
- В графе переходов автомата Мили на дугах указываются через символ «/» входные и выходные символы.
- Таблица переходов состоит из двух частей: в левой части записываются значения функции выходов, в правой части - значения функции переходов.

Построение автомата Мили

- *Пример.* Построим преобразователь, который распознает арифметические выражения, порождаемые грамматикой:
 - $S \rightarrow a+S \mid a-S \mid +S \mid -S \mid a$
- и устраняет из этих выражений избыточные унарные операции.
- Например, выражение $-a + -a - + -a$ он переведет в $-a -a +a$.

Построение автомата Мили

Входной язык является регулярным множеством.

- Пусть $M = (K, X, Y, f, g)$, где
- $K = \{q_0, q_1, q_2, q_3, q_4\}$,
- $X = \{a, +, -\}$,
- $Y = X$.

Автомат Мили для устранения избыточных унарных операций

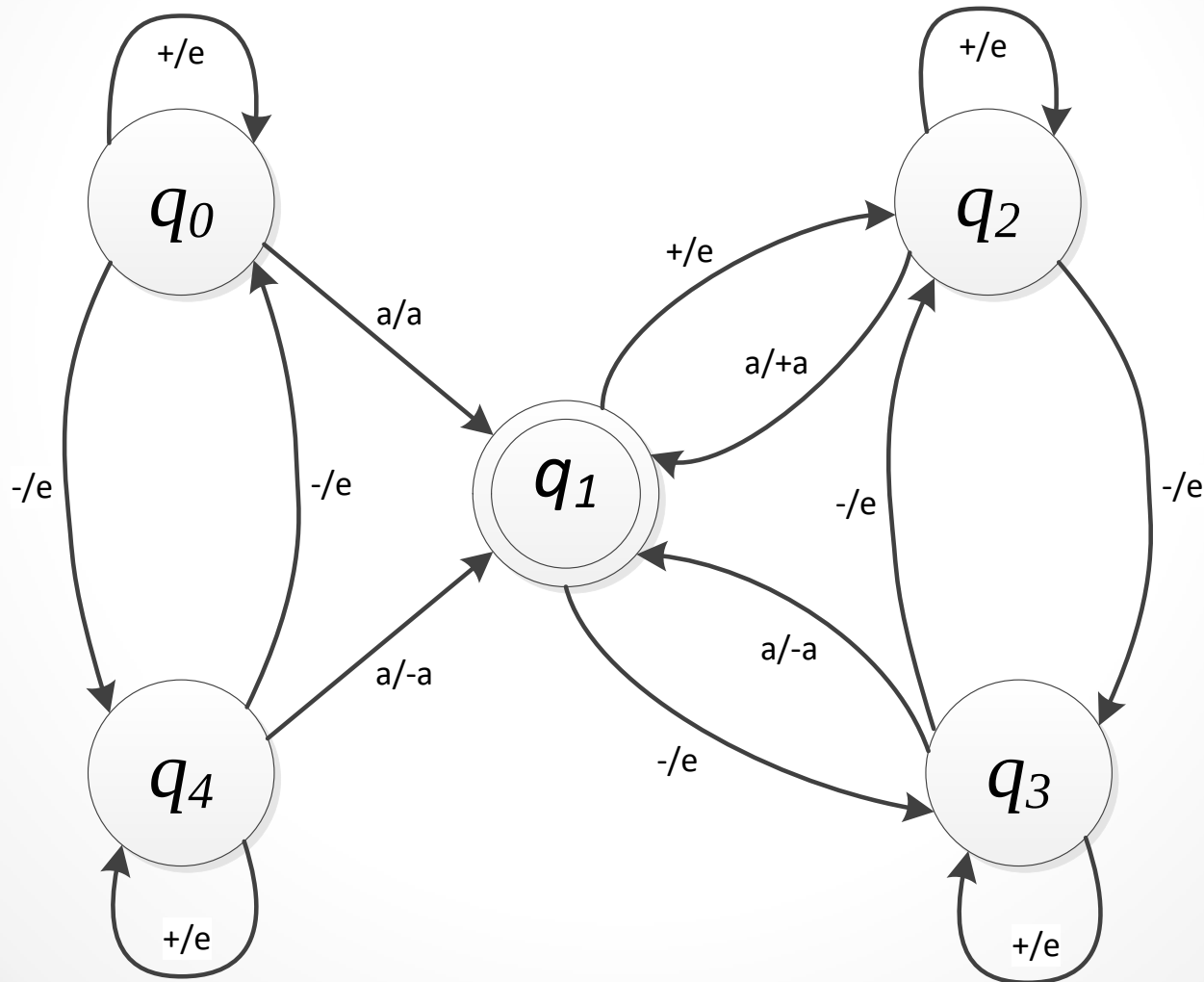


Таблица переходов автомата Мили

	Y			K		
K\X	<i>a</i>	+	-	<i>a</i>	+	-
q ₀	<i>a</i>	ε	ε	q ₁	q ₀	q ₄
q ₁	-	ε	ε	-	q ₂	q ₃
q ₂	+ <i>a</i>	ε	ε	q ₁	q ₂	q ₃
q ₃	- <i>a</i>	ε	ε	q ₁	q ₃	q ₂
q ₄	- <i>a</i>	ε	ε	q ₁	q ₄	q ₀

Автомат Мура

Автомат Мура - это «пятерка» вида $U = (K_1, X, Y, f_1, h)$, где:

- K_1 - множество состояний автомата;
- X - входной алфавит;
- Y - выходной алфавит;
- f_1 - функция переходов (отображение $K \cdot X \rightarrow K$);
- h - функция выходов (отображение $K \cdot X \rightarrow Y$).

Автомат Мура

- При представлении автомата Мура графом дуги помечаются символами входного алфавита, а каждая вершина графа - состоянием и символом выходного алфавита.
- У автомата Мура сначала порождается выход, а потом - переход в следующее состояние, причем выход определяется только состоянием автомата.



Детерминированные и недетерминированные МП-автоматы

Недетерминированный МП-автомат (НМП)

Функция переходов δ может возвращать **несколько возможных действий** для одной комбинации (состояние, символ, вершина стека).

Автомат может "угадывать" правильный путь вычисления.
Допускает ϵ -переходы в любой момент.

Мощность: Распознаёт **все контекстно-свободные языки** (КС-языки)

Детерминированный МП-автомат (ДМП)

Для любой комбинации существует **не более одного перехода**.
Если определён ϵ -переход, то не должно быть переходов по входным символам.

Работает предсказуемо, без "выбора" между альтернативами.

Мощность: Распознаёт только **подмножество КС-языков** (класс DCFL).

Критическое различие: В отличие от конечных автоматов (где ДКА и НКА эквивалентны), для магазинных автоматов НМП строго мощнее ДМП. Существуют КС-языки, распознаваемые только недетерминированными автоматами!

Пример: Язык палиндромов

Язык $L = \{w wR \mid w \in \{a,b\}^*\}$, где wR — это w в обратном порядке (например, aba , $abba$, $aabbbaa$), является классическим контекстно-свободным языком.

НМП может распознать

Недетерминированный автомат может "угадать" середину слова w , затем проверить, что вторая половина является зеркальным отображением первой, используя стек.

- Фаза 1: Читает символы и помещает в стек
- Недетерминированный выбор: "Это середина!"
- Фаза 2: Сопоставляет входные символы с символами из стека

ДМП не может распознать

Детерминированный автомат не может точно определить, где находится середина слова, без возможности "угадывания". Он не знает, когда переключиться с накопления в стеке на сопоставление.

Вывод: Язык палиндромов — это КС-язык, но не DCFL.

Этот пример демонстрирует принципиальное различие в вычислительной мощности между детерминированными и недетерминированными магазинными автоматами.

Применение и контрольные вопросы

Практическое применение

МП-автоматы находят прямое применение в **синтаксическом анализе** (парсинге) языков программирования. Они проверяют правильность вложенности скобок, операторов, блоков кода — всех конструкций с иерархической структурой.

Вопросы для самопроверки

1. Почему конечный автомат не может распознать язык $a^n b^n$?
2. По какому принципу работает стек МП-автомата?
3. Какие три параметра определяют переход в МП-автомате?
4. В чём ключевое различие между ДМП и НМП? Какой из них мощнее?

Заключение: Магазинные автоматы — это элегантное расширение конечных автоматов, добавляющее память в виде стека. Это позволяет работать с вложенными структурами произвольной глубины и распознавать контекстно-свободные языки — следующую ступень в иерархии Хомского.

Рекомендуемая литература

1. Бильгаева Н.Ц. Теория алгоритмов, формальных языков, грамматик и автоматов: Учебное пособие. Улан-Удэ: Изд-во ВСГТУ, 2000.
2. Волкова И.А., Вылиток А.А., Руденко Т.В. Формальные грамматики и языки. Элементы теории трансляции: уч. пособие для студентов II курса. - М.: Издательский отдел факультета ВМиК МГУ, 2009. - 115 с.
3. Мартыненко Б.К. Языки и трансляция: учебное пособие. - СПб: Из-во С.-Петербургского университета, 2004. - 236 с.
4. Хопкрофт Дж., Мотвани Р., Ульман Дж. Введение в теорию автоматов, языков и вычислений: Пер. с англ. - М.: Издательский дом «Вильямс», 2008. - 528 с.